

UNIVERSIDADE DO MINHO

Licenciatura em Engenharia Informática



Projeto de Processamento de Linguagens
2025/2026

Trabalho Prático

Relatório Técnico - Compilador Fortran 77 para EWVM

Beatriz Miranda, A107365

Francisco Barros, A106894

Tiago Martins, A106927

Maió 2026

Índice

1	Introdução	3
2	Arquitetura	3
2.1	Pré-processador	3
2.2	Lexer	4
2.3	Parser	5
2.4	Análise Semântica	5
2.5	Geração de IR	6
2.6	Otimizador	6
2.7	Geração de Código VM	7
3	Implementação	7
3.1	Controlo de fluxo: IF e DO	7
3.2	Declarações e tipos	8
3.3	Arrays multidimensionais	8
3.4	I/O: READ, PRINT e WRITE	8
3.5	Subprogramas: FUNCTION e SUBROUTINE	8
3.6	Exemplos representativos	8
4	Testes e Validação	9
5	Conclusão	10
A	Gramática	11
B	Instruções de execução	11
B.1	Preparação	11
B.2	Compilação	11
B.3	Testes	11

Lista de Figuras

1	Pipeline do compilador (visão geral).	3
2	Excerto de IR (TAC) gerado.	6
3	Excerto de código VM gerado.	7
4	Resultado da suite de testes no ambiente local.	10

1 Introdução

Este relatório tem como objetivo apresentar o trabalho prático desenvolvido na unidade curricular de Processamento de Linguagens. O trabalho consiste no desenvolvimento de um compilador para um *subset* de Fortran 77, tendo como destino final a máquina virtual disponibilizada (EWVM). Deste modo, apresentamos a arquitetura do sistema, as decisões de implementação tomadas pelo grupo, as dificuldades encontradas e os resultados obtidos.

Para além da implementação base, o projeto inclui funcionalidades de valorização, como suporte a subprogramas externos (FUNCTION, SUBROUTINE, CALL e RETURN), tratados por *inlining* na geração de IR, e tratamento robusto de arrays multidimensionais com indexação dinâmica. Na iteração final foram ainda reforçados o pré-processamento de entrada, a compatibilidade com programas em formato livre simples, o *backend* de I/O, a validação semântica de labels e ciclos, a linearização correta de arrays segundo a ordem de Fortran e a cobertura dos testes *end-to-end*. O objetivo deste documento é apresentar, de forma estruturada, todas as fases da *pipeline* de compilação, bem como as opções arquiteturais que sustentam a entrega.

2 Arquitetura

A implementação foi organizada como uma *pipeline* de compilação com fronteiras claras entre fases, de forma a separar responsabilidades e reduzir acoplamento entre componentes. Em vez de concentrar toda a lógica num único módulo, cada etapa transforma uma representação de entrada numa representação de saída mais estruturada, permitindo testar e validar o compilador de forma incremental.

Em termos práticos, o fluxo segue a sequência:

1. normalização do código Fortran 77 em *fixed-form* ou formato livre simples;
2. tokenização do código normalizado;
3. construção da AST;
4. validação semântica;
5. geração de representação intermédia;
6. otimização;
7. emissão de código EWVM.

A principal vantagem desta arquitetura é que cada módulo pode evoluir de forma relativamente independente. Por exemplo, melhorias no parser não obrigam a alterar o pré-processamento, desde que o contrato entre as duas fases seja preservado. Do mesmo modo, a fase de otimização opera sobre IR e não depende diretamente de detalhes da sintaxe concreta da linguagem fonte.

Outra decisão importante foi preservar informação contextual útil (como número de linha e labels) ao longo das primeiras fases. Esta informação é essencial para a semântica (por exemplo, validação de GOTO) e para mensagens de erro mais informativas.

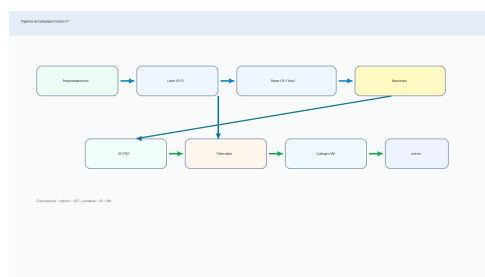


Figura 1: Pipeline do compilador (visão geral).

2.1 Pré-processor

O primeiro componente executado é o **preprocessor.py**, responsável por normalizar o texto fonte antes da análise léxica. O compilador aceita o formato clássico *fixed-form* de Fortran 77 e, adicionalmente, um formato livre simples, compatível com programas copiados diretamente do enunciado. Esta opção reduz o risco de rejeitar

exemplos corretos apenas por diferenças de alinhamento, sem abandonar a semântica de labels característica de Fortran 77.

A normalização segue as regras tradicionais:

- colunas 1–5: campo de label;
- coluna 6: marca de continuação;
- colunas 7–72: código efetivo da instrução.

Com base nestas regras, e nas extensões aceites para formato livre, o módulo executa quatro tarefas essenciais:

1. remove linhas de comentário e linhas vazias;
2. identifica labels e separa-as do conteúdo da instrução;
3. junta corretamente linhas de continuação quando a coluna 6 indica continuação;
4. remove comentários de fim de linha, preservando corretamente texto entre aspas.

O resultado não é apenas texto limpo: cada linha lógica passa a ser representada com metadados (número de linha original, label e conteúdo normalizado). Esta escolha facilita as fases seguintes, que podem operar sobre uma estrutura uniforme e já livre de ruído lexical.

A manutenção da interpretação *fixed-form* é importante porque preserva a convenção histórica de colunas e a detecção posicional de labels de DO/CONTINUE. O suporte a formato livre simples é uma camada de conveniência controlada: permite linhas como `PROGRAM T, 20 IF ( . . . ) THEN` ou `PRINT *, X`, mas continua a produzir a mesma representação lógica usada pelo resto da *pipeline*.

Em resumo, o pré-processador funciona como uma fase de estabilização da entrada: reduz ambiguidades precoces e garante que o lexer recebe uma sequência previsível de linhas lógicas.

2.2 Lexer

A segunda fase é implementada em **lexer.py**, usando `ply.lex`. O objetivo é converter cada linha lógica normalizada numa sequência de tokens tipados, já preparados para análise sintática.

A implementação define explicitamente:

- palavras reservadas da linguagem (`PROGRAM`, `IF`, `DO`, `READ`, `CALL`, etc.);
- literais (`INT_LIT`, `REAL_LIT`, `BOOL_LIT`, `STRING_LIT`);
- operadores aritméticos, relacionais e lógicos;
- separadores e símbolos estruturais.

Do ponto de vista de robustez, duas decisões foram relevantes:

1. **case-insensitive**: o lexer é construído para aceitar diferentes combinações de maiúsculas/minúsculas, refletindo o uso típico de Fortran;
2. **normalização de identificadores**: os identificadores são normalizados internamente para maiúsculas, garantindo consistência nas fases seguintes.

A tokenização é feita por linha lógica e preserva o contexto de origem (linha e label). Assim, o parser não recebe apenas tokens isolados, mas sim uma sequência contextualizada de (`lineno`, `label`, `token_stream`).

Em termos de cobertura da linguagem, esta fase já resolve vários pontos críticos:

- operadores pontuados de Fortran (`.EQ.`, `.NE.`, `.AND.`, `.OR.`, `.NOT.`);
- literais reais com notação científica, incluindo expoente E e D;
- strings delimitadas por aspas simples ou duplas, incluindo apóstrofes duplicados no estilo Fortran;
- formas alternativas de palavras-chave relevantes, como `GO TO`;
- distinção entre palavras reservadas e identificadores válidos.

Quando surge um caractere inválido, o lexer sinaliza erro imediatamente, evitando propagar estados inconsistentes para o parser. Esta estratégia reduz o custo de depuração e melhora a previsibilidade da pipeline.

2.3 Parser

O **parser.py** segue uma abordagem híbrida, combinando `ply.yacc` com análise estrutural por linhas. Esta decisão foi tomada para conciliar dois objetivos: cumprir o requisito de utilização de `PLY` e, ao mesmo tempo, lidar bem com construções clássicas de Fortran 77 orientadas a label.

A parte suportada diretamente por `ply.yacc` cobre:

- expressões aritméticas, relacionais e lógicas com precedência/associatividade;
- instruções de linha única (declarações, atribuições, `PRINT`, `READ`, `WRITE`, `GOTO`, `GO TO`, `CALL`, `RETURN`, `STOP`);
- declarações `CHARACTER` simples e com tamanho explícito, como `CHARACTER*12`.

Já as estruturas de bloco multi-linha são tratadas por controlo estrutural explícito:

- `IF (...) THEN ... ELSE ... ENDIF`, incluindo a grafia `END IF`;
- `DO label var = start, end [, step] ... label CONTINUE`.

Esta separação traz vantagens práticas:

1. mantém a gramática LALR mais simples e focada nas construções locais;
2. evita complexidade artificial para padrões de bloco dependentes de labels;
3. melhora a qualidade das mensagens de erro em estruturas incompletas (por exemplo, `IF` sem `ENDIF` ou `DO` sem fecho).

Outro aspeto importante é a desambiguação entre acesso a array e chamada de função em formas do tipo `ID(...)`. O parser faz esta decisão com base no contexto conhecido de subprogramas e funções intrínsecas, produzindo nós AST distintos (`ArrayRefNode` ou `FuncCallNode`) conforme o caso. Nesta fase é também suportada a forma `F()`, permitindo funções sem argumentos.

Para evitar ruído numa utilização normal do compilador, a construção dos parsers `PLY` é feita com `NullLogger`. Assim, avisos internos sobre tokens que pertencem a outra gramática parcial não aparecem no terminal durante a compilação de exemplos, mantendo o CLI limpo para demonstração.

No final da fase sintática, o resultado é uma AST completa, com corpo principal e subprogramas externos, pronta para validação semântica. Assim, o parser funciona como ponte entre uma entrada ainda textual (tokens por linha) e uma representação estrutural rica, adequada às fases de análise e geração de código.

2.4 Análise Semântica

O módulo **semantic.py** implementa as validações necessárias para garantir a corretude do programa antes da geração de código. Esta fase atua sobre a AST e usa uma tabela de símbolos por escopos, separando o programa principal dos subprogramas (`FUNCTION` e `SUBROUTINE`).

A análise é feita de forma estruturada: primeiro são registadas assinaturas globais dos subprogramas (nome, tipo de retorno, parâmetros), e depois cada corpo é validado no seu escopo próprio. Esta estratégia evita ambiguidades em chamadas e permite validar referências cruzadas com antecedência.

Durante o percurso da AST, são verificadas as seguintes propriedades:

- **declaração e uso de identificadores:** cada variável, array ou subprograma deve estar declarado antes de ser usado no respetivo escopo;
- **compatibilidade de tipos:** atribuições e operações são validadas para evitar combinações inválidas; quando aplicável, são aceites promoções seguras (por exemplo, de inteiro para real);
- **validação de operadores:** operadores aritméticos exigem operandos numéricos; `.AND.`, `.OR.` e `.NOT.` exigem operandos lógicos; comparações ordenadas exigem operandos numéricos; `**` é aceite apenas com expoente inteiro; esta validação impede combinações inválidas como `LOGICAL + INTEGER` ou `CHARACTER * INTEGER`;
- **condições lógicas:** expressões de controlo em `IF` têm de produzir valor lógico;
- **labels e desvio:** cada `GOTO` é validado contra labels realmente existentes no bloco analisado e labels duplicadas são rejeitadas;

- **regras de ciclos DO:** variável de controlo, limites e passo são verificados com as restrições semânticas da linguagem, incluindo rejeição de passo literal zero;
- **arrays:** são validadas dimensões, rank, tipo dos índices e acessos fora da forma esperada;
- **subprogramas:** número, tipo e categoria dos argumentos (escalar/array) devem coincidir com a assinatura de FUNCTION/SUBROUTINE.

Em caso de erro, o compilador interrompe a pipeline nesta fase, evitando que um programa semanticamente inválido avance para IR ou *backend*. Esta opção melhora a depuração, uma vez que concentra a deteção de incoerências na fase em que a informação semântica ainda é mais rica.

2.5 Geração de IR

Representação intermédia usada A linguagem intermédia usada nesta fase é **TAC** (*Three-Address Code*). Só depois da otimização em TAC é gerado o código final para a **EWVM**.

O módulo `ir_gen.py` é responsável pelo *lowering* da AST para uma representação intermédia em TAC (*Three-Address Code*). Esta representação desacopla o *front-end* do compilador do *backend*, facilitando a otimização e a posterior geração de código VM.

Cada instrução TAC é modelada com operador e operandos (`op`, `result`, `arg1`, `arg2`), o que torna explícita a dependência entre cálculos e controlo de fluxo. Ao longo desta fase são gerados temporários e labels para linearizar estruturas de alto nível em passos elementares.

A tradução cobre, entre outros, os seguintes casos:

- **expressões:** operadores aritméticos, relacionais e lógicos são convertidos em sequências TAC com temporários;
- **atribuições e I/O:** leitura, escrita e cópia de valores são normalizadas em operações simples;
- **controlo de fluxo:** IF/ELSE e DO são convertidos para labels e saltos condicionais/incondicionais;
- **arrays:** acessos e atualizações tornam-se operações explícitas de carga e armazenamento indexado;
- **subprogramas:** chamadas são baixadas para IR por inlining, com preservação da semântica através de mapeamento de símbolos, etiquetas internas, renomeação sistemática de variáveis locais e saltos de retorno.

No caso de PRINT/WRITE list-directed, a IR inclui separadores explícitos entre valores, permitindo que uma instrução como PRINT *, A, B produza uma saída legível equivalente a A B, em vez de concatenar diretamente os valores. Nos ciclos DO, a geração de IR emite também a label real associada ao CONTINUE terminal, garantindo que um GOTO para essa label tem destino válido no código VM.

Ao final desta etapa, o compilador deixa de depender da forma sintática original do programa: toda a informação relevante passa a estar codificada numa sequência TAC uniforme, pronta para otimização e geração de código final.



Figura 2: Excerto de IR (TAC) gerado.

2.6 Otimizador

O módulo `optimizer.py` aplica um conjunto de *passes* de otimização sobre o IR antes da geração de código final para a VM, executando-os em sequência até atingir **ponto fixo**, isto é, até duas iterações consecutivas produzirem exatamente o mesmo IR. Os passes implementados são: *constant folding*, que avalia operações com operandos constantes em tempo de compilação; *copy propagation*, que propaga aliases introduzidos por atribuições simples

para reduzir temporários intermédios; eliminação de temporários mortos, que remove definições de `_t*` cujo valor nunca é utilizado; eliminação de código inalcançável, que remove instruções após saltos incondicionais; eliminação de escritas mortas, que remove atribuições sobrescritas antes de serem lidas; e simplificações *peephole*, que eliminam cópias redundantes (`x = x`) e saltos para a label imediatamente seguinte.

Adicionalmente, o *constant folding* foi refinado para ficar coerente com a semântica efetiva da máquina/gerador de código: a operação DIV segue a semântica da EWVM (divisão inteira quando ambos os operandos são inteiros e divisão real quando pelo menos um dos operandos é real) e a operação POW respeita a estratégia do backend (multiplicação sucessiva enquanto o expoente for positivo), permitindo pré-calcular casos constantes simples sem alterar o significado do programa. Foram também incluídas dobras de constantes para operações unárias e lógicas/relacionais frequentes (NEG, NOT, AND, OR e comparações), reduzindo temporários e instruções redundantes no IR antes da geração de código final.

2.7 Geração de Código VM

O módulo `codegen.py` traduz o IR (TAC) para as instruções da EWVM. O *backend* implementa as seguintes funcionalidades:

- mapeamento de variáveis para memória global;
- linearização de arrays multidimensionais em ordem *column-major*, coerente com Fortran;
- suporte a índices constantes e dinâmicos;
- I/O tipado no código gerado, com separadores em escrita list-directed;
- emissão segura de literais string no código VM;
- tradução do operador de exponenciação `**` para um ciclo VM de multiplicações sucessivas (instrução intermédia POW).

O operador `**` merece nota especial. Em vez de depender de uma instrução nativa de potência na EWVM, o backend emite um ciclo que inicializa o resultado a 1 e multiplica pela base enquanto o expoente inteiro for positivo. Esta solução mantém a tradução portátil dentro do conjunto de instruções já usado pelo projeto.

Para arrays, o backend distingue dois casos. Quando todos os índices são literais inteiros, o deslocamento é resolvido estaticamente e o código usa diretamente PUSHG/STOREG. Quando algum índice é dinâmico, o deslocamento linear é calculado em tempo de execução e são emitidas instruções de acesso indireto (LOADN/STOREN). Esta combinação melhora a eficiência nos casos constantes e preserva a generalidade nos casos com variáveis de índice.

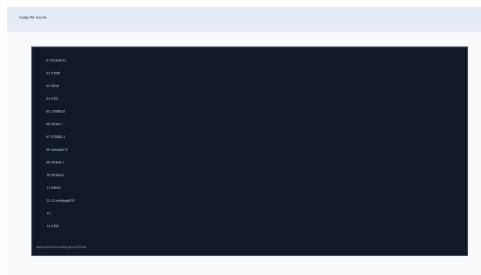


Figura 3: Excerto de código VM gerado.

3 Implementação

Concluída a visão geral da arquitetura, este capítulo aprofunda as decisões de implementação mais relevantes, analisando de que modo as diferentes funcionalidades do *subset* de Fortran 77 foram tratadas ao longo da pipeline.

3.1 Controlo de fluxo: IF e DO

Uma das principais dificuldades do Fortran 77 em *fixed-form* é a gestão de blocos com labels. As construções IF/ENDIF e DO/.../CONTINUE não seguem uma estrutura de blocos delimitados por chavetas ou palavras-chave simples, mas sim por labels numéricas associadas a instruções CONTINUE.

A solução adotada foi o *parser* estrutural por linhas tokenizadas, que percorre a sequência de linhas e identifica os delimitadores de bloco com base nas labels. A validação semântica complementa esta análise, garantindo a coerência das labels (e.g., ausência de GOTO para labels inexistentes, estruturas DO sem CONTINUE correspondente).

Para a geração de IR, cada bloco de controle é emitido com labels únicas, evitando colisões em programas com múltiplos ciclos ou condicionais. A versão final emite ainda o rótulo VM correspondente ao CONTINUE terminal do ciclo DO, uma vez que Fortran permite saltos explícitos para esse label. Sem esta emissão, programas sintaticamente válidos com GOTO para o fim do ciclo poderiam falhar apenas na execução da VM.

3.2 Declarações e tipos

As declarações de variáveis são processadas durante a análise semântica, que constrói uma tabela de símbolos por âmbito (programa principal e cada subprograma). A compatibilidade de tipos é verificada em atribuições e expressões aritméticas, com emissão de erros descritivos para facilitar a depuração por parte do utilizador.

A versão atual reforça ainda a validação ao nível dos operadores: cada operador exige operandos do tipo correto, o que impede combinações semanticamente inválidas mesmo que a sintaxe esteja bem formada. Foram também acrescentadas validações para labels duplicadas e para DO com passo literal zero. Esta validação aproxima o compilador do contrato semântico esperado de uma linguagem tipada.

3.3 Arrays multidimensionais

O suporte a arrays multidimensionais implicou duas decisões de implementação relevantes. Em termos de semântica, as dimensões declaradas são validadas contra os acessos no código. Em termos de *backend*, os arrays são linearizados por *stride*: o endereço de cada elemento é calculado em tempo de compilação para índices constantes, e emitido como sequência de instruções aritméticas para índices dinâmicos.

A linearização segue a ordem *column-major* de Fortran, isto é, o primeiro índice é o que varia mais rapidamente. Por exemplo, em $A(2,3)$, os elementos $A(1,2)$ e $A(2,1)$ têm deslocamentos diferentes dos que seriam obtidos numa ordem típica de C. Esta correção é essencial para compatibilidade semântica com Fortran 77.

3.4 I/O: READ, PRINT e WRITE

As instruções READ, PRINT e WRITE são traduzidas para as primitivas de I/O da EWVM. O *backend* seleciona a instrução adequada consoante o tipo da expressão (inteiro, real, lógico ou string), garantindo que o código gerado realiza a leitura e escrita com o formato correto. A escrita list-directed introduz separadores entre valores, evitando saídas ambíguas como 12 quando o programa pretende escrever 1 e 2 como campos distintos.

3.5 Subprogramas: FUNCTION e SUBROUTINE

O suporte a subprogramas externos constitui a principal funcionalidade de valorização do projeto. A implementação introduz desafios em três fases distintas.

Na análise semântica, são validadas as assinaturas e os argumentos de cada chamada, assim como o tipo de retorno das FUNCTIONS. No *lowering* para IR, cada subprograma é inlinado com um mapeamento interno de símbolos e labels, evitando colisões de nomes em múltiplas chamadas.

Na versão final, o inlining foi corrigido para isolar corretamente variáveis locais. Anteriormente, apenas parâmetros e valores de retorno recebiam nomes internos únicos; uma variável local com o mesmo nome de uma variável do programa principal podia causar colisão silenciosa. Agora, as declarações locais do subprograma são renomeadas com prefixos únicos por chamada e emitidas no IR através de instruções declarativas. Assim, preserva-se a noção de âmbito mesmo sem implementar uma *stack* de chamadas nativa na EWVM. O comando RETURN é materializado como salto para um rótulo de retorno, ficando o backend encarregado apenas de emitir a sequência TAC já linearizada.

Esta decisão é semanticamente importante: a transformação por inlining só é correta se o código resultante for equivalente ao programa original. A renomeação sistemática de símbolos locais garante precisamente essa equivalência.

3.6 Exemplos representativos

Tal como no relatório de referência, a profundidade técnica beneficia de exemplos concretos que mostram o percurso entre o código fonte, a AST, o IR e o código final para a EWVM. Os casos que se seguem são representativos das principais decisões de implementação do compilador.

Programa elemental: HELLO O exemplo mais simples valida o funcionamento de ponta a ponta da pipeline sem interferência de controlo de fluxo ou estruturas compostas.

```
PROGRAM HELLO
PRINT *, 'Ola, Mundo!'
END
```

O código gerado é compacto e directo, confirmando a tradução base da instrução PRINT:

```
PUSHN 0
START
PUSHS "Ola, Mundo!"
WRITES
WRITELN
STOP
```

Subprogramas: CONVERTOR O exemplo CONVERTOR ilustra a integração entre o programa principal e uma função externa, bem como o uso de MOD, IF e GOTO no interior do subprograma.

```
PROGRAM CONVERTOR
INTEGER NUM, BASE, RESULT, CONVRT
READ *, NUM
DO 10 BASE = 2, 9
RESULT = CONVRT(NUM, BASE)
PRINT *, RESULT
10 CONTINUE
END

INTEGER FUNCTION CONVRT(N, B)
INTEGER N, B, QUOT, REM, POT, VAL
VAL = 0
POT = 1
QUOT = N
20 IF (QUOT .GT. 0) THEN
REM = MOD(QUOT, B)
VAL = VAL + (REM * POT)
QUOT = QUOT / B
POT = POT * 10
GOTO 20
ENDIF
CONVRT = VAL
RETURN
END
```

Este caso é particularmente relevante porque confirma a estratégia de *inlining* adotada para subprogramas. A função é validada semanticamente, convertida para IR com etiquetas de retorno e símbolos locais renomeados e, no final, integrada no fluxo principal sem necessidade de uma convenção de chamada nativa na EWVM.

4 Testes e Validação

A validação do compilador inclui testes unitários e de pipeline que cobrem todas as fases da compilação.

Os testes de integração com a EWVM, que anteriormente poderiam depender de ligação à máquina de referência, correm localmente graças ao módulo **vm.py**, um interpretador Python do subconjunto de instruções EWVM gerado pelo compilador. Esta solução elimina a dependência de rede nos testes automáticos, mantendo a cobertura *end-to-end* completa.

Para além disso, o módulo **optimize.py** disponibiliza uma interface de linha de comandos para otimizar representações IR textuais independentemente do compilador, útil para depuração e validação isolada do otimizador.

Foram acrescentados testes específicos para os pontos de maior risco de desvalorização: programas em formato livre simples, linhas com indentação leve, END IF, READ(*,*), WRITE(*,*), separação de valores em PRINT, labels duplicadas, passo zero em DO e ordem *column-major* em arrays multidimensionais.

Foram ainda acrescentados testes baseados em propriedades com a biblioteca **Hypothesis**, que verificam automaticamente invariantes do compilador com centenas de entradas geradas — nomeadamente que o otimizador não altera o *output* do programa e que operações aritméticas, divisão inteira e MOD produzem resultados consistentes com a semântica esperada.

```

(.venv) (base) beatriz@beatriz:~/PL/PROJETO/PL-G225 python -m pytest -q
===== tests coverage ===== [100%]
coverage: platform linux, python 3.10.12-final-0

Name                Stmts  Miss  Cover   Missing
-----
src/ast_nodes.py      92      0  100%
src/codegen.py       362     45   88%   37, 90-101, 118, 129, 148, 157, 195, 201-202, 223-224, 229, 235, 239, 245, 262, 264-265, 305, 405-406, 425-438,
446, 453-454, 465, 474
src/ewm.py            57     57    0%   8-110
src/ir.py             17      0  100%
src/ir_gen.py        257     15   94%   105-106, 135, 273-275, 286-287, 303-305, 314-316, 340
src/lexer.py          65      1   98%   132
src/main.py           82     82    0%   1-113
src/optimize.py       97     21   78%   29, 31, 66, 68, 70, 81, 112-114, 118-123, 127-136
src/optimizer.py     288     22   92%   41, 44, 68, 82-83, 100-101, 109, 113, 115, 120-122, 190, 228, 252, 323, 332, 361, 391, 401-402
src/parser.py         383     35   88%   216, 241, 246, 251, 275-277, 312, 322, 348, 354, 357, 366, 381, 384, 388, 398, 400, 409, 419, 443, 451, 454, 45
6, 459, 467, 469, 472, 479, 483, 514, 522
src/preprocessor.py   71      5   93%   24, 34, 40, 51, 54
src/repl.py          152    152    0%   14-209
src/semantic.py      291     38   87%   66-67, 96-97, 188, 200, 210, 238-239, 250-251, 268, 270, 282, 290, 294, 302, 333, 348, 355-358, 375-380, 385-38
6, 391-394, 398, 414-417, 421, 437
src/symbol_table.py   19      2   89%   19, 23
src/visualizer.py     132    132    0%   11-228
src/vm.py             163     23   86%   20, 42, 50, 55, 60, 85-86, 129, 131, 138, 146, 149-150, 161-162, 169-170, 175, 181, 188-189, 196, 214

TOTAL                2448     630   74%
Coverage HTML written to dir htmlcov
Required test coverage of 70% reached. Total coverage: 74.26%
103 passed in 8.19s

```

Figura 4: Resultado da suite de testes no ambiente local.

5 Conclusão

Concluindo este projeto com sucesso, o grupo considera que o desenvolvimento do mesmo constituiu uma excelente oportunidade para aplicar na prática os conceitos de processamento de linguagens, desde a análise léxica até à geração de código para uma máquina virtual real.

O projeto apresenta uma implementação modular e consistente de um compilador Fortran 77 para EWVM, com cobertura dos requisitos base e dos principais pontos de valorização. A pipeline completa, o desenho por fases e a validação automatizada sustentam uma entrega tecnicamente sólida e uma defesa bem fundamentada. Em particular, na versão final foram reforçados o pré-processamento, o suporte a variantes sintáticas usuais, o backend de I/O, a validação semântica de labels e ciclos, a linearização de arrays multidimensionais segundo a ordem de Fortran e a cobertura dos testes *end-to-end*.

A estrutura adotada, separação clara entre *front-end* (preprocessamento, lexer, parser e semântica), camada intermédia (IR/TAC e otimização) e *backend* (geração VM), melhora a manutenibilidade, facilita a depuração por fases e permite introduzir melhorias de forma incremental sem comprometer a estabilidade global do compilador.

Os resultados obtidos na suite de testes (103 passed) demonstram robustez nos casos essenciais do enunciado e nos cenários de valorização, incluindo subprogramas e arrays com indexação dinâmica. Em contexto de defesa, isto permite sustentar não apenas que o compilador funciona, mas também que as decisões de implementação foram tecnicamente justificadas, testáveis e alinhadas com critérios de correção, estrutura e eficiência.

Ao longo do desenvolvimento surgiram vários desafios técnicos. A gestão de blocos com labels em Fortran 77 *fixed-form* revelou-se a maior dificuldade sintática: as construções DO/CONTINUE e IF/ENDIF dependem de labels numéricas em vez de delimitadores explícitos, o que torna a gramática LALR pura inadequada para o efeito. A solução passou por combinar `ply.yacc` com controlo estrutural por linhas. No otimizador, a *copy propagation* produzia inicialmente resultados incorretos em programas com ciclos, substituindo variáveis de controlo pelos seus valores iniciais, e a solução foi invalidar aliases em cada ponto de junção. Por fim, a compatibilidade com a EWVM exigiu ajustes no formato de labels e literais, que eram rejeitados pela VM nas primeiras versões.

A Gramática

```
program      := "PROGRAM" ID stmt* "END" subprogram*
subprogram   := function_def | subroutine_def
function_def := type "FUNCTION" ID "(" [id_list] ")" stmt* "END"
subroutine_def := "SUBROUTINE" ID "(" [id_list] ")" stmt* "END"

stmt         := decl
              | assign | print | read
              | if_stmt | do_stmt | goto
              | continue | call | return | stop

decl         := type var_spec ("," var_spec)*
type         := "INTEGER" | "REAL" | "LOGICAL" | "CHARACTER"
              | "CHARACTER" "*" INT_LIT
var_spec     := ID | ID "(" expr_list ")"
assign       := lvalue "=" expr
lvalue       := ID | ID "(" expr_list ")"
print        := "PRINT" "*" "," expr_list
              | "WRITE" "*" "," expr_list
              | "WRITE" "(" "*" "," "*" ")" expr_list
read         := "READ" "*" "," lvalue_list
              | "READ" "(" "*" "," "*" ")" lvalue_list
if_stmt      := "IF" "(" expr ")" "THEN" stmt* ["ELSE" stmt*] ("ENDIF" | "END" "IF")
do_stmt      := "DO" label ID "=" expr "," expr ["expr"]
              stmt* label "CONTINUE"
goto         := "GOTO" (INT_LIT | ID)
              | "GO" "TO" (INT_LIT | ID)
call         := "CALL" ID "(" [expr_list] ")"

expr         := literal | ID | ID "(" [expr_list] ")"
              | "(" expr ")" | "-" expr | ".NOT." expr
              | expr ("+" | "-" | "*" | "/" | "**") expr
              | expr (".EQ." | ".NE." | ".LT." | ".LE." | ".GT." | ".GE.") expr
              | expr (".AND." | ".OR.") expr
```

B Instruções de execução

B.1 Preparação

```
make install
```

B.2 Compilação

```
make compile
```

Ou um ficheiro específico:

```
python3 src/main.py examples/fatorial.f -o out.vm
python3 src/main.py examples/fatorial.f --dump-ast --dump-ir
```

B.3 Testes

```
make test
```